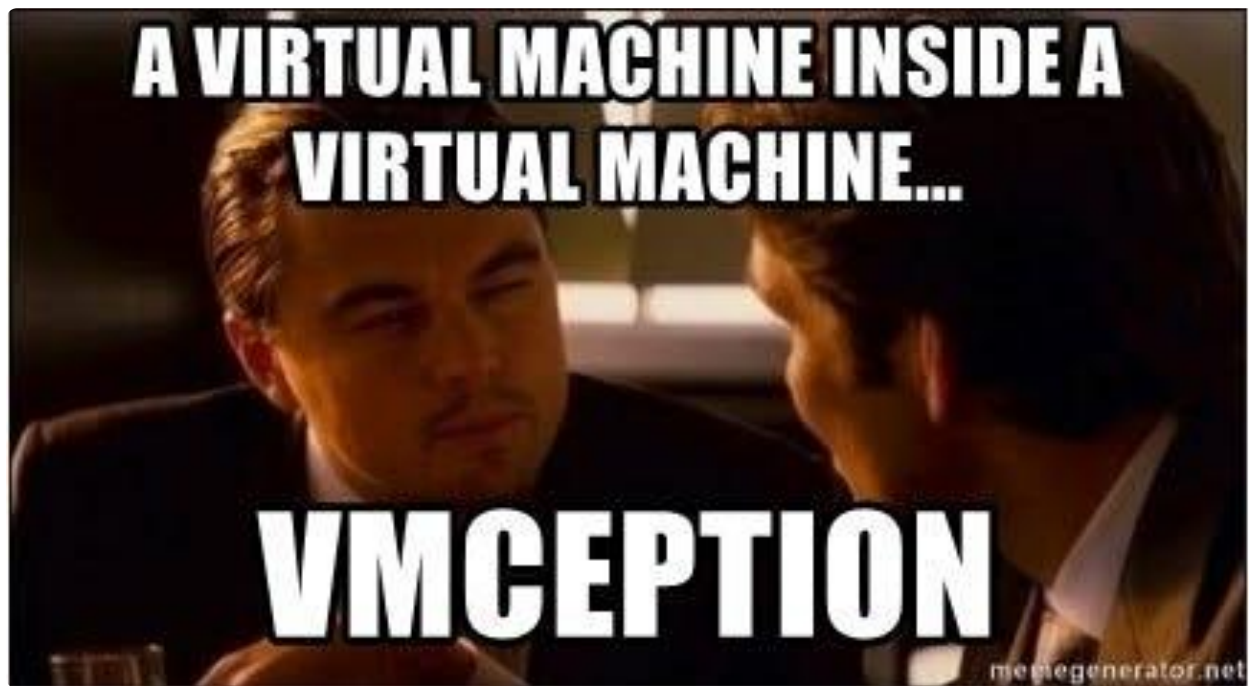




PVM (Python virtual machine):

As we read in the previous blog, named Looking inside the Python code!! which was information about how exactly when a python code is...

panutanu R

Published March 20, 2021

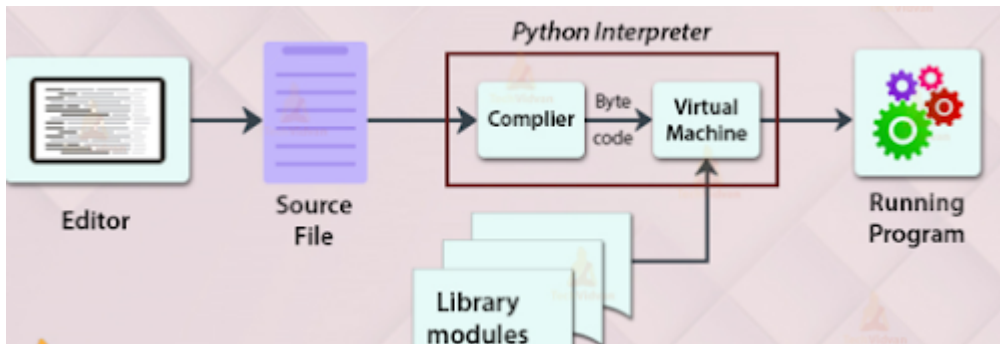
Updated January 7, 2022

Free: Yes

[Read on Freedium](#) · [original](#)

As we read in the previous blog, named [Looking inside the Python code!!](https://www.blogger.com/u/0/blog/post/edit/6337451913408280043/3362459998531695769?hl=en) (https://www.blogger.com/u/0/blog/post/edit/6337451913408280043/3362459998531695769?hl=en) which was information about how exactly when a python code is written, its execution happens, in brief. So, we have the code compiled first and then it is interpreted. The compilation is been hidden from the user, as we saw information about CPython. We also referred to this in a diagram which indicated that once the Python code is compiled it is been converted to the Bytecode and then the interpreter will take this Bytecode as input and give the output which would be machine code.

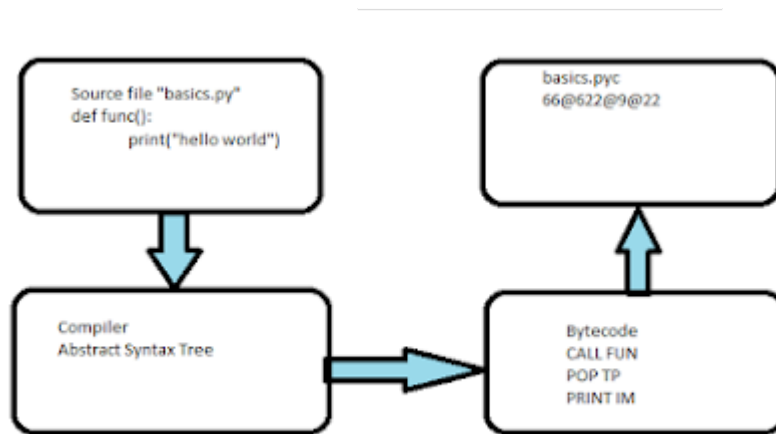
If you know about how the compilation process in C works, we can relate it as the compiler after doing all the syntax analysis, Lexical analysis, and stuff, convert the code into the assembly language and then the Assembler converts this same language into the machine level language. Here PVM can be correlated to the Assembler. We will discuss the compiler in upcoming blogs.



To understand more about the PVM we need to understand its architecture.

The architecture comprises of four important parts:

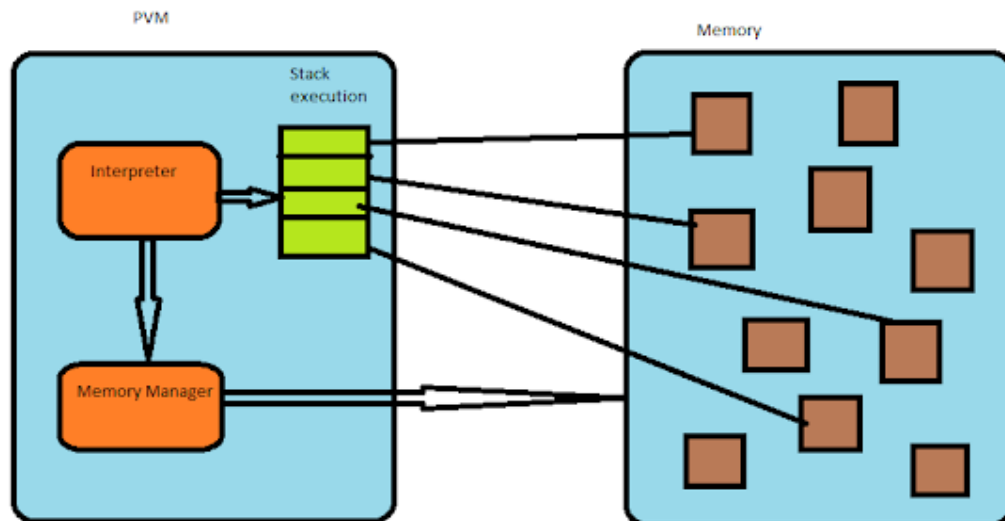
- 1] Compiler
- 2] Interpreter
- 3] Memory manager
- 4] Stack



From the above figure, let us consider we have a file named "basics.py". This is the source file, now the compiler will take this file and do its lexical analysis, syntax analysis, semantics analysis, and then bytecode generation is done. Provided there are no errors in any of the processes. In Bytecode, it can be compared to instructions in assembly language.

Later, in this Bytecode, if we check in any editor, it would be like certain garbage values as the editor won't understand the bytecode.

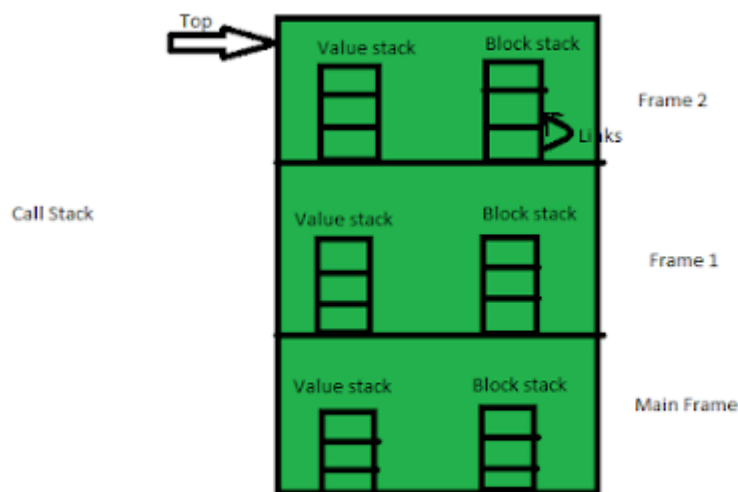
After these states, the interpreter will now come into the picture.



After the generation of the bytecode from the source code now PVM functioning will start. We have an interpreter, which will take the Bytecode and call the memory manager via some functions or APIs and ask to create a memory object in the memory section. Using some other API's Memory manager will create these

objects. Also, the interpreter will create a stack for the execution and the memories for the instruction set will be referenced which was the prior task of the memory manager.

So, now in the stack execution, we have something called the Call stack. It is in the frame format with the Value stack and Block stack being part of each frame. As we saw earlier we have a memory manager which is responsible for making the links for a memory object. For the stack in each frame, it will have allocated memory references pointing to these memory objects.



Let us now consider the call stack as shown in the above figure. In a python program, we have the whole program itself which is regarded as the Main frame. We have certain functions which we write in this code, which form the other frames as we see in the above figure. Frame 1, is basically the first function which is been written in the python program file. We also have a top for this stack, so that the program execution can happen linearly with respect to the frame calls. Once the execution of a function is completed the frame will be popped out. Now, inside every frame there formulates the value stack and the block stack. Value stack is the one where variables are been operated, pushed, and popped as per the Bytecode instruction. Also known as the evaluation stack or the data stack.

In each frame, there's a block stack. This is used by the frame to keep track of certain types of control structures: loops, try and except blocks are the entries to be pushed onto the block stack, and the block stack gets popped whenever you exit one of those structures. This helps the frame in the execution to link to know active execution and if it requires a certain part of the memory link. If a local or global variable has to be accessed this can be done in accordance and the reference from the block stack.

Frames will have a reference to the code object of the function and references to the local and global variables. Most importantly it will have the reference to the builtins. These are the ones that are already a part of python.

The Object class, which happens to be the base class for all Python objects, is defined in this module. All built-in data type classes such as numbers, string, list, etc are defined in this module. The Base-Exception class, as well as all built-in exceptions, are also defined in it. A Value stack and reference to the top of the value stack, Block stack, and its reference. Exception info carriers otherwise also known as the run time errors like type, value, and traceback where exactly was the error occurrence. Most importantly a reference to the previous frame, this is because, consider function1 calls function2, then after the successful execution of the function2 the program flow must return to function1.

In further blogs we will try to read more about the Python programming basics just a brief overview of code implementation.